

Rapport exercice V

I Présentation de l'exercice

Le but de l'exercice était d'implémenter un algorithme qui augmente le contraste d'une image en niveau de gris, et ensuite de le paralléliser.

II Calcule du temps

Afin de calculer les temps d'exécutions, je me suis aidé d'un script python, et de la sortie standard du programme en c.

Tout les données sont donc automatiquement compiler dans un fichier Json.

Pour une bonne cohérence au niveau des mesures, j'ai fait une moyenne sur 100 exécutions pour chaque test.

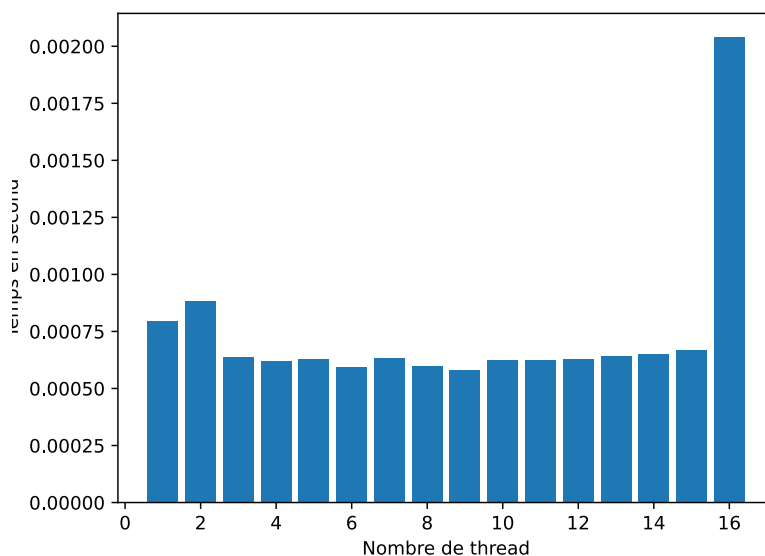
III Résultats

Voici les résultats en fonction de chacune des images données en test, avec un nombre de threads allant de 1 à 16.

III.1 Temps pour la première image

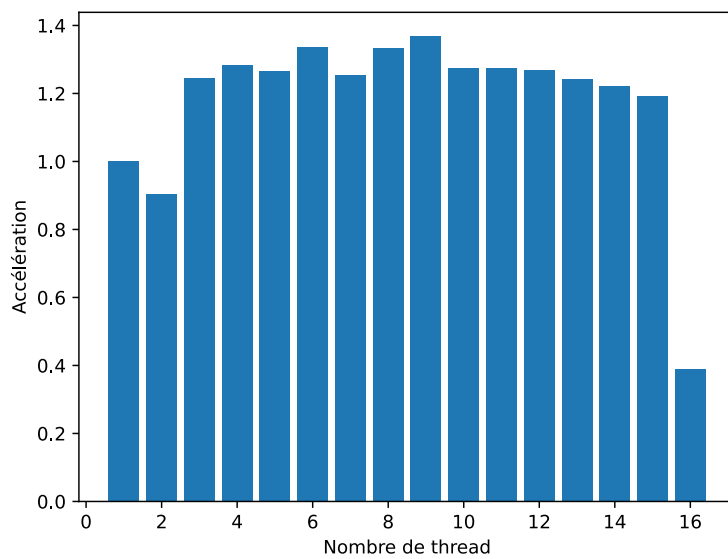
Nombre de thread	Temps pris
1	0.79626ms
2	0.88134ms
3	0.63897ms
4	0.621009ms
5	0.62836ms
6	0.59617ms
7	0.63549ms
8	0.5971ms

Nombre de thread	Temps pris
9	0.58117ms
10	0.62412ms
11	0.62496ms
12	0.62721ms
13	0.64116ms
14	0.65119ms
15	0.667429ms
16	2.04206ms



La forte augmentation du temps à la fin est certainement dû à un problème de cohérence de cache.

Voici maintenant un graphique synthétisant l'accélération en fonction du nombre de threads :

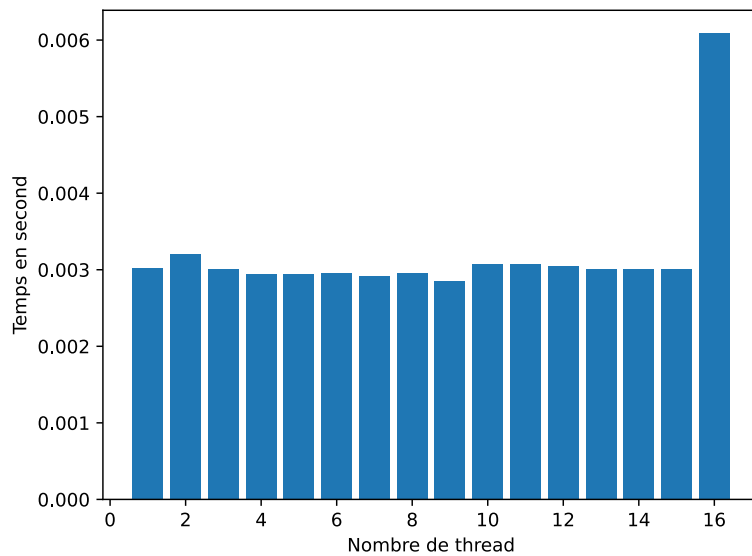


L'accélération maximal est de 1.37, pour 10 threads.

III.2 Temps pour la seconde image

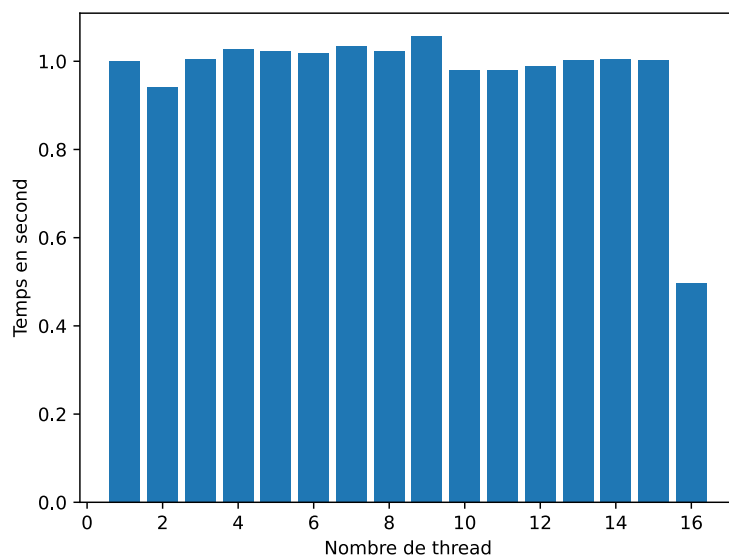
Nombre de thread	Temps pris
1	3.01692ms
2	3.20601ms
3	3.00252ms
4	2.939089ms
5	2.948889ms
6	2.96244ms
7	2.91737ms
8	2.95102ms

Nombre de thread	Temps pris
9	2.856219ms
10	3.077049ms
11	3.07956ms
12	3.04823ms
13	3.006349ms
14	3.002679ms
15	3.00686ms
16	6.084829ms



Encore une fois, on observe une forte augmentation du temps pour 16 threads, sans doute pour les mêmes raisons.

Voici maintenant un graphique synthétisant l'accélération en fonction du nombre de threads :



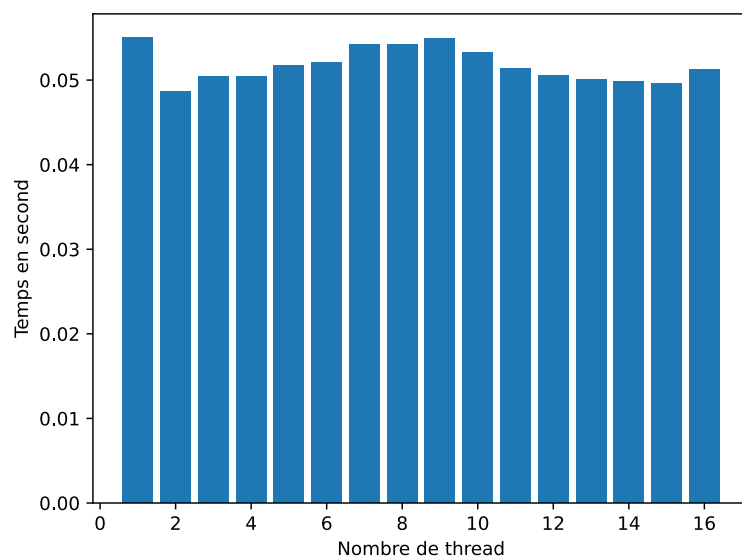
L'accélération maximale est de 1.06, pour 9 threads.

III.3 Temps pour la troisième image

Nombre de thread	Temps pris
1	55.072409ms
2	48.595439ms
3	50.45896ms
4	50.43615ms

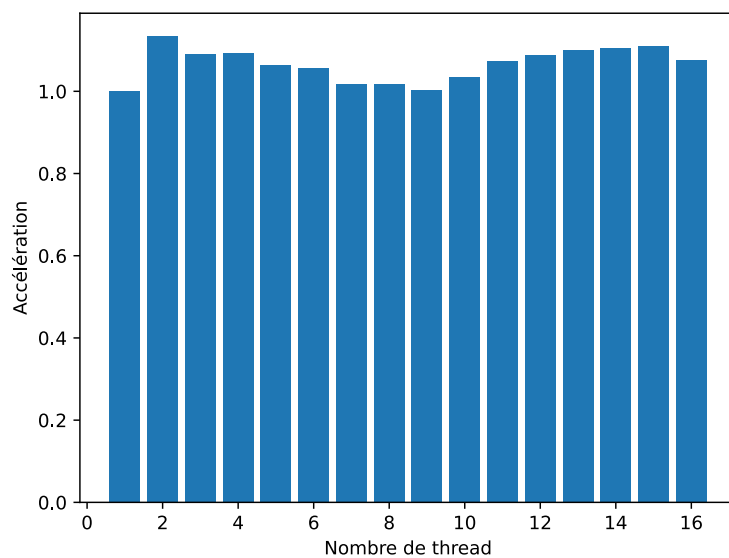
Nombre de thread	Temps pris
5	51.733179ms
6	52.092369ms
7	54.18325ms
8	54.17084ms

Nombre de thread	Temps pris
9	54.971509ms
10	53.253929ms
11	51.34808ms
12	50.589919ms
13	50.02549ms
14	49.852489ms
15	49.58174ms
16	51.237679ms



Pour cette image nous n'observons pas d'augmentation significative.

Les données étant assez grandes, les problèmes de cohérences de cache sont alors évités.



L'accélération maximal est de 1.13, pour 2 threads, globalement, l'accélération semble resté stable, et surtout supérieur à 1.

IV Annexe

IV.1 Machine utilisé

Tous les temps ont été enregistré sur un ordinateur portable avec un processeur x86-64 de huit cœurs physique dont deux virtuel par cœurs, cadencé à ~3.5GHz (sur secteur).

IV.2 Données récolter

Toutes les mesures effectuées sont stockées dans un document *json* récupéré grâce à la sortie standard du programme, et a un script python, trouvable sur le [GitHub](#).

IV.3 Code

L'entièreté du code se trouve sur le [GitHub](#) dans le fichier Ex5.